

COGNIZANT NPN

Retail Demand Forecasting - Walmart M5

Problem Approach, Algorithms, Accuracy Strategy & Hierarchical Implementation
Plan

What this document gives you

A practical, technically defensible route from the exact uploaded M5 files to a 28-day forecasting system. It explains what to build first, which algorithms to test, which techniques are likely to produce the strongest accuracy, how to evaluate fairly, and how to organize the complete hierarchy-aware solution.

Important accuracy note

There is no single algorithm that can be honestly guaranteed to be “most accurate” before validation. For this dataset, the strongest evidence from the original M5 competition and related research supports gradient-boosted trees - especially LightGBM - combined with rich lag/rolling features, direct/recursive multi-horizon forecasting, and hierarchy-aware alignment or reconciliation. The right final model should be selected by time-based validation using WRMSSE and supporting metrics.

1. The recommended approach at a glance

Your Cognizant statement asks for item-level demand forecasts for 28 days while handling hierarchical aggregation, external covariates such as price/promotion/holiday information, and intermittent demand. The uploaded dataset gives you the ingredients to do this: historical sales, calendar/event features, weekly prices, and the required 28-day output format.

Recommended project strategy

Build a strong baseline first, then a LightGBM-based global forecasting model with time, lag, rolling, price and event features. Validate with rolling time splits. Add intermittent-demand features and hierarchical reconciliation/alignment. Finally compare against a small set of alternative models and blend the best complementary forecasts. Keep the solution explainable enough for a Cognizant presentation.

1. Understand hierarchy	2. Reshape data	3. EDA & data quality	4. Feature engineering
5. Time-based validation	6. Baselines + ML models	7. Hierarchical reconciliation	8. 28-day forecasting
9. WRMSSE + MAE/RMSE	10. Ensemble	11. Business dashboard	12. Presentation/demo

2. What the problem actually requires

Forecast the target

Predict daily demand for each item-store series for the next 28 days.

Use the hierarchy

The data has state, store, category, department and item relationships. The final system should not ignore these relationships.

Use external covariates

Use calendar, event, SNAP and price information. The uploaded dataset does not provide a clean explicit "stockout" flag, so stockouts must be treated as a data-quality/modeling issue rather than assumed directly from a zero sale.

Handle intermittent demand

Many retail series contain many zeros and occasional positive sales. Features and models should be designed for sparse demand.

Evaluate correctly

Do not use random train/test splitting. Forecasting validation must respect time order and should focus on the competition metric WRMSSE, with MAE/RMSE as supporting diagnostics.

3. Dataset-aware solution design

sales_train_validation.csv	Primary historical daily sales used to train and create validation windows	Reshape d_1...d_1913 to long format; build target and lag features.
sales_train_evaluation.csv	Extended historical sales containing the additional evaluation horizon in the supplied dataset	Used to understand/evaluate the 28-day future period and mirror the competition setup.
calendar.csv	Actual dates, week IDs, weekday, month, year, events and SNAP flags	Join on d; derive time/event features.
sell_prices.csv	Weekly price by store and item	Join on store_id + item_id + wm_yr_wk; derive price level/change/discount proxies.
sample_submission.csv	Required prediction layout	Generate id + F1...F28 in the exact final format.

The most important data transformation

Do not model directly from one row containing d_1...d_1913 as hundreds of separate target columns. Convert the sales table from wide to long format so one observation represents one item-store-day. This makes lag, rolling, date and price features natural.

4. Data preparation and EDA - step by step

A. Validate identities

Check unique items, departments, categories, stores and states; verify each item belongs to one department/category and each store belongs to one state.

B. Convert dates

Map d_1, d_2, ... to actual calendar dates through calendar.csv.

C. Join prices

Merge weekly sell_price using store_id + item_id + wm_yr_wk. Preserve missing price as a meaningful state and investigate why it is missing.

D. Check missingness

Separate true missing values from zeros. A zero in sales means zero observed units; it is not automatically the same as missing demand or a stockout.

E. Explore intermittent demand

Compute zero-demand share, average interval between non-zero sales, mean positive demand and recent demand activity for each series.

F. Explore seasonality

Measure day-of-week, monthly, yearly, holiday and event patterns.

G. Explore price response

Look at sales behavior at different price levels and price changes. Do not claim causal elasticity from observational data alone.

H. Identify data leakage risks

Every feature available to the model must be computable at forecast time. Never use future sales to build features for validation.

5. Feature engineering - what to create

Calendar	dow, weekend, month, quarter, year, week-of-year, days to/from event	Captures recurring seasonality and known calendar effects	★★★★★
Sales lags	lag_1, lag_7, lag_14, lag_21, lag_28, lag_56, lag_364/365	Captures short-term memory, weekly cycles and year-over-year patterns	★★★★★
Rolling statistics	mean/sum/std/min/max over 7,14,28,56 days; shifted before calculation	Summarizes recent level, volatility and trend	★★★★★
Same-weekday history	Average/median over recent same weekdays	Useful when Monday behaves differently from Saturday	★★★★☆
Trend features	28-day vs 7-day mean; ratio to 4-week mean; slope-like signals	Shows acceleration/deceleration in demand	★★★★☆
Price features	current price, previous price, price change, price ratio to recent average	Captures price effects and shifts	★★★★☆
Event/SNAP	event flags/types, lead/lag windows, state-specific SNAP flags	Captures predictable demand changes	★★★★☆
Intermittency	zero-streak length, days since last sale, recent nonzero count, positive-demand mean	Helps the model understand sparse demand	★★★★★
Hierarchy/context	item/store/category/department/state categorical features	Lets one global model learn shared patterns	★★★★★

Golden rule for all lag/rolling features

Always shift the feature window so it only uses information known before the forecast origin. For example, a 7-day mean used to predict tomorrow should be based on the previous 7 observed days, not a window containing tomorrow's target.

6. Algorithms and techniques to test

Naive seasonal baseline	Forecast last observed value or same weekday/previous week	Very fast, interpretable, required to prove the ML model adds value	Baseline only
Moving-average / seasonal mean	Simple statistical benchmark	Strong sanity check for stable series	Baseline only
Croston / SBA / TSB	Intermittent-demand methods	Designed for sparse/zero-heavy series	Useful benchmark or specialized component
XGBoost	Gradient-boosted tree regression	Strong nonlinear tabular learner; can exploit lag/features	High

LightGBM	Gradient-boosted tree regression	Fast, scalable, handles nonlinear interactions and many engineered features; closely aligned with strong M5 solutions	Very high - primary candidate
CatBoost	Gradient boosting with categorical handling	Convenient for categorical hierarchy variables	High alternative
N-BEATS	Deep time-series model	Strong on higher-level smoother/continuous series; useful in hybrid hierarchy setups	Advanced hybrid
DeepAR / TFT	Deep probabilistic / multi-series forecasting	Can model multiple series jointly and provide rich nonlinear dynamics	Advanced, higher complexity
Ensemble / blending	Combine complementary models	Often improves robustness when models make different errors	Very high - final layer

Best practical starting point

For your team, start with LightGBM as the main global model, not LSTM. Use rich lag/rolling/calendar/price/intermittency features, predict the four weeks recursively or with horizon-specific models, and then add hierarchy reconciliation/alignment. This is both technically strong and much easier to explain than jumping directly to a large deep-learning stack.

7. How to maximize accuracy - a realistic priority order

“Precision” is not the right single word for a forecasting competition. A forecast can be assessed with MAE or RMSE, but the original M5 Accuracy competition used WRMSSE, which weights errors across the full hierarchy. Therefore, your goal should be to minimize the forecasting error under a competition-style evaluation rather than chase an arbitrary percentage accuracy.

Priority	Technique	What to do	Impact
1	Time-aware validation	Use rolling-origin validation with multiple 28-day holdouts.	Non-negotiable
2	Feature engineering	Lags + rolling stats + weekday/yearly patterns + price/event/SNAP + intermittency features.	Highest practical impact
3	LightGBM tuning	Tune depth/leaves, learning rate, min data in leaf, feature fraction, bagging and regularization.	Very high
4	Horizon strategy	Compare one model per horizon vs direct blocks vs recursive weekly forecasting.	High
5	Hierarchy alignment	Reconcile or align bottom-level forecasts so aggregate totals are coherent.	High for stated problem
6	Model blending	Blend the best diverse models using weights learned on validation forecasts.	High
7	Special handling for sparse series	Use intermittency features and/or a specialized model for very sparse series.	Useful
8	Post-processing	Clip negatives to zero; consider conservative shrinkage for unstable series; preserve non-negative demand.	Useful

Evidence from the original M5 competition

A published review of the M5 Accuracy challenge reports that the winning submission used multiple direct and recursive LightGBM models. A second-place method combined LightGBM for granular intermittent series, N-BEATS for higher-level continuous series, and hierarchical alignment. A separate strong implementation also used LightGBM with past-transaction features. These results support using LightGBM as your core model and making the hierarchy explicit rather than treating the task as plain one-series forecasting.

What NOT to promise

Do not say “LightGBM gives 99% accuracy” or “this algorithm is guaranteed to be the most accurate.” The correct statement is: “We will benchmark several approaches on rolling 28-day validation windows and select/blend the models that minimize WRMSSE, with MAE/RMSE as supporting metrics.”

8. Proper hierarchical approach - step by step

Level structure in the M5 data

State -> Store -> Category -> Department -> Item, with the final forecasting series defined at item-store level. The competition evaluates not only the lowest-level series but also aggregated series across the hierarchy. Your Cognizant solution should therefore forecast at the detailed level and then ensure aggregate coherence/alignment.

Step 1 - Define the bottom level

Use each item-store series as the base forecasting unit.

Step 2 - Build the aggregation map

Create deterministic mappings for state/store and category/department/item relationships.

Step 3 - Train global bottom-level model

Train LightGBM using item/store/category/department/state features plus lag/rolling/calendar/price/event/intermittency features.

Step 4 - Produce 28-day bottom forecasts

Generate F1...F28 for all item-store series without leakage.

Step 5 - Aggregate forecasts

Sum item-store forecasts to store, department, category, state and other required levels.

Step 6 - Check coherence

Ensure child totals add up to parent totals after aggregation.

Step 7 - Reconcile or align

Compare simple bottom-up totals against a reconciliation/alignment method. Choose the version with better validation WRMSSE.

Step 8 - Blend if useful

Blend aligned LightGBM forecasts with a complementary model for the series where it improves validation.

Step 9 - Evaluate at all levels

Calculate WRMSSE overall and inspect error by hierarchy level/store/category to identify where the model is weak.

Step 10 - Generate final submission

Write exactly id + F1...F28 using the sample_submission layout.

9. Validation design - the part teams often get wrong

Why random train/test split is wrong

If you randomly shuffle daily rows, the model can accidentally learn patterns from dates that occur after the validation period. That is temporal leakage. Forecasting must simulate the real situation: the model sees the past and predicts a future block.

Recommended rolling-origin setup

Validation window	Rule
Window A	Train up to T-56; validate next 28 days.
Window B	Train up to T-84; validate next 28 days.
Window C	Train up to T-112; validate next 28 days.
Final holdout	Reserve the most recent known 28-day block for final internal comparison before using the supplied evaluation horizon.

Metrics

Metric	Use	Interpretation
WRMSSE	Primary competition-style metric	Lower is better; evaluates scaled errors across the hierarchy with weights.
MAE	Easy-to-explain diagnostic	Average absolute unit error.
RMSE	Penalizes large misses	Useful for detecting occasional large forecast failures.
WMAE / weighted diagnostics	Business diagnostic	Useful if your team wants to weight important series/store combinations.
Zero-demand diagnostics	Intermittency check	Track false positive demand and missed spikes for sparse items.

10. Recommended end-to-end architecture

1. RAW M5 FILES

sales + calendar + prices

2. DATA LAYER

validate IDs -> reshape wide to long -> join calendar/price

3. EDA LAYER

sales distribution -> zero demand -> seasonality -> price/event effects

4. FEATURE LAYER

lags -> rolling -> calendar -> price -> event/SNAP -> intermittency -> hierarchy

5. MODEL LAYER

Naive + LightGBM + XGBoost/CatBoost -> optional N-BEATS/TFT

6. FORECAST LAYER

direct/recursive 28-day predictions

7. HIERARCHY LAYER

aggregate -> reconcile/align -> check coherence

8. EVALUATION

WRMSSE + MAE + RMSE across rolling holdouts

9. FINAL OUTPUT

sample_submission.csv format + dashboard + business insights

11. How your 8-member team can divide the work

1	Problem + business	Problem definition, objective, business impact, 28-day target	Why forecasting matters and what success means
2	Dataset + EDA	Dataset profiling, hierarchy counts, distributions, visualizations	Every file and key column
3	Preprocessing	Long-format pipeline, joins, missingness/zero analysis	No leakage; correct joins
4	Hierarchy	Aggregation map, bottom-up + reconciliation/alignment experiments	Why hierarchy matters and how coherence works
5	Feature engineering	Lag/rolling/calendar/price/event/intermittency features	Why each feature exists
6	Modeling	Baselines + LightGBM + alternative model	Why LightGBM, how training works
7	Validation + metrics	Rolling splits, WRMSSE, MAE/RMSE, error analysis	Why random split is invalid
8	Dashboard + demo	Forecast UI, charts, business recommendations, final integration	How a manager would use the output

Team rule

Every member should understand the full pipeline at a high level. Each member can be the primary expert for one module, but nobody should present a module they cannot defend in technical questioning.

12. A practical build roadmap for your team

Phase	Workstream	Output
Phase 1	Dataset inspection	Load all 5 files, profile shape, IDs, missingness, duplicates, target distribution.
Phase 2	Long-format data pipeline	Convert wide sales to item-store-day rows and join calendar + price.
Phase 3	EDA	Produce demand, zero-demand, seasonality, store/category and price/event analyses.
Phase 4	Baseline	Implement naive/seasonal forecasts and calculate your local validation metrics.
Phase 5	LightGBM v1	Build a global model using core lag/rolling/calendar features.
Phase 6	LightGBM v2	Add price/event/SNAP/intermittency features and tune hyperparameters.
Phase 7	Hierarchy	Add aggregation, coherence checks and reconciliation/alignment experiment.
Phase 8	Ensemble	Blend best complementary forecasts using validation data only.
Phase 9	Final 28-day run	Retrain using all allowed historical data; generate F1...F28.
Phase 10	Demo	Streamlit/dashboard, plots, business interpretation, presentation and mock Q&A;

Recommended final model for a student team

Primary: LightGBM global model. Features: lags, shifted rolling stats, day/week/month/year, same-weekday history, price changes, event/SNAP flags, intermittency features and categorical hierarchy IDs. Forecasting: compare direct horizon models against weekly recursive forecasting. Hierarchy: aggregate bottom forecasts and test reconciliation/alignment. Final: blend the best validated models. This gives a strong balance of accuracy, runtime, explainability and presentation quality.

13. Common mistakes to avoid

Using random train-test split

Creates temporal leakage and produces unrealistic validation scores.

Training an LSTM first

Deep learning is not automatically more accurate. Strong tree-based baselines are easier to debug and were extremely competitive in M5.

Ignoring hierarchy

This misses a central requirement of the statement and can lead to inconsistent aggregate forecasts.

Treating every zero as a stockout

The supplied files do not contain a clean stockout indicator. Zero sales are observed demand, not proof of unavailable inventory.

Using future information in rolling features

This is direct leakage. Every feature for day t must be computable using information available before t .

Optimizing only MAE

A low MAE does not guarantee a strong WRMSSE score. Evaluate the hierarchy-aware metric used by the M5 competition.

Overcomplicating the stack

A clear LightGBM + hierarchy + validation + dashboard solution is better than a complicated model nobody can explain.

14. Final blueprint - what I recommend you build

1

Use the supplied M5 CSVs as the single source of truth.

2

Reshape sales into item-store-day format and join calendar and weekly price data.

3

Perform EDA focused on seasonality, intermittent demand, price effects and hierarchy.

4

Create leak-free lag, rolling, calendar, price, event/SNAP and intermittency features.

5

Build naive baselines first.

6

Train LightGBM as the main global forecasting model.

7

Compare XGBoost/CatBoost and, if your team has enough time, a deep or specialized intermittent-demand model.

8

Use rolling 28-day validation windows.

9

Optimize the model for WRMSSE while reporting MAE and RMSE for understandable diagnostics.

10

Aggregate forecasts and test reconciliation/alignment to satisfy the hierarchy requirement.

11

Blend complementary models only when validation proves the blend is better.

12

Generate the final 28-day F1-F28 forecasts and present them through a clear dashboard.

13

Explain business impact: inventory planning, stockout risk, excess inventory and promotion/event-aware planning.

Bottom line

For this exact Cognizant/M5 task, do not begin by asking “Which deep-learning algorithm is the most advanced?” Begin by asking “What validation setup matches the 28-day forecasting problem and the hierarchy?” Then build a strong LightGBM global model around leak-free temporal features, add hierarchy-aware reconciliation/alignment, and use model blending only when it improves validation. That is the most defensible route to a high-quality result.

Evidence and references

The original M5 Accuracy challenge evaluated 30,490 bottom-level item-store series and 42,840 total series after aggregation, using WRMSSE. Published analyses report winning/near-winning approaches centered on LightGBM, direct/recursive forecasting, and hierarchy-aware combinations. One second-place method combined LightGBM for granular intermittent series with N-BEATS for higher-level continuous series and hierarchical alignment. These sources are used here to justify the recommended algorithm hierarchy, not to claim a guaranteed score for your project.

Additional references: M5 evaluation methodology and competition analysis; published work discussing intermittent demand and the limits of standard point-forecast objectives.